

# A Hyper-Heuristic Framework for Adaptive Task Scheduling in Heterogeneous Cloud Environments

Project Report

MSc Research Project  
Cloud Computing

**Olumide Akinkunmi**  
Student ID: 23351411

School of Computing  
National College of Ireland

Supervisor: Yasantha Samarawickrama

July 2026

## Abstract

Heterogeneous cloud computing environments require efficient task scheduling to handle volatile workloads, cut operational costs, and reduce carbon footprints. Static heuristics like HEFT and Min-Min work well under specific conditions, but they fail when workload dynamics change or system faults occur. This report presents a dynamic Multi-Armed Bandit (MAB) hyper-heuristic framework for cloud task scheduling. The architecture combines a Python controller with a Cloud-Sim Plus simulation backend using standard I/O streaming pipes. The controller manages a low-level heuristic pool of nine algorithms, including classical list schedulers, meta-heuristics, and local search methods.

We evaluate four MAB selection mechanisms:  $\epsilon$ -Greedy, Standard UCB1, Discounted UCB, and Scaled Thompson Sampling. The framework uses a workload fingerprinting method based on the Coefficient of Variation (CoV) alongside a batch-relative multi-objective reward function balancing makespan, execution cost, carbon emissions, and resource utilization. We test the system using HPC2N and NASA real-world trace datasets. Experimental results show that Discounted UCB adapts best to non-stationary environments; and when subjected to a 90% capacity drop, Discounted UCB decayed legacy performance memory and shifted policy from meta-heuristics to PEFT within 10 rounds, whereas Standard UCB remained stuck in stale Q-values. Overall, the hyper-heuristic framework achieved up to a 34.3% reduction in both operational costs and carbon emissions compared to traditional baselines.

# 1 Introduction

## 1.1 Background and Motivation

Data centers are the backbone of modern digital infrastructure, as they continuously process massive streams of diverse workloads every second. Managing these resources requires efficient task scheduling to assign arriving tasks to heterogeneous Virtual Machines (VMs) without violating SLA agreements, execution constraints or underutilizing hardware. However, finding an optimal schedule in a heterogeneous cloud environment is a well-known NP-hard optimization problem. As system scale increases and workload dynamics shift, traditional scheduling strategies struggle to maintain performance.

This challenge has become a much more pressing issue with the recent rise of Artificial Intelligence (AI) and Machine Learning (ML) workloads. With the rapid adoption of generative AI, large language models, and continuous deep learning pipelines, there has been an unprecedented demand for cloud compute infrastructure. Data centers now face dense batch jobs, erratic compute spikes, and heterogeneous hardware requirements that combine memory-bound and compute-heavy tasks. This surging compute demand drives up operational costs and drastically increases energy consumption and carbon emissions. Consequently, modern data center managers must balance multi-objective trade-offs, aiming to minimize makespan and financial costs while simultaneously curbing carbon footprints and maintaining high VM utilization.

## 1.2 Problem Statement

Static Low-Level Heuristics (LLHs) like Heterogeneous Earliest Finish Time (HEFT), PEFT, Min-Min, and Sufferage rely on fixed rules about task characteristics and re-

source availability. For instance, Min-Min performs well when short tasks dominate, but causes severe blocking when resource-intensive jobs enter the queue. Meta-heuristics like Simulated Annealing can explore complex search spaces effectively, but their high computational complexity makes them too slow for fast streaming workloads.

To add to these issues, cloud environments are inherently non-stationary; workload arrival rates change rapidly, trace volatility fluctuates, and unexpected infrastructure faults or virtual machine drops can suddenly dampen processing capacity. Static scheduling rules cannot adapt on the fly when these environmental disruptions happen. When an algorithm fails to adjust to shifting conditions, the data center suffers from resource underutilization, higher operational expenditure, and avoidable carbon emissions.

### 1.3 Proposed Solution

This project presents a dynamic Multi-Armed Bandit (MAB) hyper-heuristic framework for task scheduling heterogeneous cloud environments. Instead of searching the task-to-VM space directly, the hyper-heuristic works at a higher level of abstraction: it dynamically selects the best-performing low-level heuristic from a pool of nine diverse algorithms based on real-time execution feedback.

The framework pairs a high-level Python decision engine with a CloudSim Plus simulation engine using persistent standard I/O streaming pipes. To handle volatile and non-stationary conditions, we evaluate four distinct MAB algorithms:  $\epsilon$ -Greedy, Standard UCB1, Discounted UCB (D-UCB), and Scaled Thompson Sampling. The framework integrates a workload fingerprinting mechanism that uses the Coefficient of Variation (CoV) to continuously measure trace volatility. It also uses a batch-relative, multi-objective reward function that squashes execution metrics into a balanced signal combining makespan, monetary cost, carbon emissions, and VM utilization.

### 1.4 Key Contributions

The main contributions of this dissertation are:

- **Decoupled Hyper-Heuristic Architecture:** We build a lightweight, streaming Python-CloudSim Plus framework that selects low-level heuristics dynamically without interrupting execution streams.
- **Non-Stationary Adaptation under Failure:** We demonstrate that Discounted UCB outperforms standard bandit variants during abrupt system failures, pivoting its policy from computationally intensive meta-heuristics to PEFT within 10 rounds after a 90% VM capacity loss.
- **Multi-Objective Carbon-Aware Reward Model:** We design a batch-relative reward function using a hyperbolic tangent ( $\tanh$ ) transformation to balance makespan, financial cost, carbon emissions, and resource utilization without scale distortion.
- **Trace-Driven Empirical Validation:** We evaluate our framework using real-world HPC2N and NASA workload traces across 10-seed multi-seed runs, achieving up to a 34.3% reduction in both monetary costs and carbon footprint compared to traditional baselines.

## 1.5 Report Structure

The rest of this report is organized as follows: Section 2 reviews related literature on cloud task scheduling, multi-armed bandits, and sustainable hyper-heuristics. Section 3 provides the theoretical foundation for our MAB formulations and reward functions. Section 4 presents the system design and pipeline architecture. Section 5 details the software implementation and IPC integration. Section 6 presents experimental results, sensitivity analyses, and statistical evaluations. Finally, Section 7 concludes the report and discusses directions for future work.

## 2 Related Work

Task scheduling in heterogeneous cloud datacenters has evolved from deterministic list-based prioritization to complex online learning frameworks. Traditional scheduling research focused on optimizing single-objective operational bounds, primarily makespan or throughput, using static low-level heuristics (LLHs) like HEFT and PEFT (Hosseini Shirvani and Noorian Talouki; 2021). While computationally efficient with  $O(V^2E)$  complexity, these fixed-rule schedulers rely on static resource availability assumptions and suffer severe performance degradation when confronted with high workload variance or abrupt hardware faults. To address these search limitations, meta-heuristics like Genetic Algorithms and hybrid swarm optimizers were introduced (Alkaam et al.; 2025). Although meta-heuristics navigate multi-objective trade-offs effectively, their high computational overhead makes them unsuitable for real-time streaming task execution.

Recent advances have shifted toward data-driven paradigms, leveraging Deep Reinforcement Learning (DRL) and Multi-Armed Bandit (MAB) formulations to enable online adaptability (De Carvalho and Sichman; 2025; Khan et al.; 2025). DRL architectures achieve high optimization bounds in stationary environments; but they face significant practical hurdles: prolonged convergence times, high training overheads, and poor zero-shot generalization when faced with sudden distribution shifts (Choppara and Lokesh; 2025; Sravan and Shaik; 2026). Conversely, MAB hyper-heuristics offer lightweight online learning by dynamically selecting optimal LLHs based on real-time execution feedback rather than managing the continuous schedule space directly (Singh et al.; 2024).

Despite these advances, three critical research gaps persist in the current literature:

1. **Historical Bias in Non-Stationary Environments:** Standard stationary MAB algorithms (such as UCB1 and standard Thompson Sampling) accumulate unweighted performance history. When unexpected node failures or capacity drops occur, these algorithms remain trapped in stale Q-values, causing prolonged adaptation lag before pivoting selection policies.
2. **Lack of Lightweight Contextual Workload Fingerprinting:** Existing contextual or content-aware schedulers often rely on heavy offline classifiers or neural feature extractors (Adil et al.; 2022). There is a lack of real-time, lightweight statistical fingerprinting mechanisms capable of dynamically modulating exploration parameters without introducing execution latency.
3. **Unbalanced Multi-Objective Reward Formulations:** While sustainability and carbon intensity have emerged as first-class scheduling parameters alongside

cost and latency (Arora et al.; 2023), existing multi-objective reward functions often suffer from scale distortion across heterogeneous execution batches, leading to single-metric dominance during bandit update steps.

By integrating Discounted UCB (D-UCB) memory decay, Coefficient of Variation (CoV) trace fingerprinting, and a batch-relative, tanh-squashed multi-objective reward model, this approach achieves rapid adaptation under infrastructure failures while jointly optimizing runtime efficiency, operational cost, VM utilization, and carbon footprint.

The remainder of this section critically reviews prior contributions across these foundational domains.

## 2.1 Cloud Task Scheduling Fundamentals

While cloud computing delivers scalable, on-demand access to computing resources, the efficient allocation of tasks to virtual machines (VMs) still poses an engineering challenge: scheduling algorithms must balance competing objectives — makespan, cost, resource utilization, and increasingly, energy and carbon emissions, while operating under dynamic and uncertain workload conditions. Because the scheduling problem is formally NP-hard, exact polynomial-time solutions do not exist, motivating the extensive use of heuristic, meta-heuristic, and learning-based approaches.

Algorithms in this domain are broadly classified into mathematical modelling (e.g. Integer Linear Programming), list heuristics, meta-heuristics, machine learning, and hybrid methods. Mathew et al. (2014) provide an early taxonomy of cloud scheduling strategies, categorizing them by performance parameters such as makespan, load balancing, and throughput. Saurav and Benedict (2021) further expanded this taxonomy by evaluating energy-aware workflow schedulers across large-scale heterogeneous architectures, highlighting the trade-offs between processing speed and energy consumption.

## 2.2 List-based and Heuristic Scheduling

List-based schedulers such as HEFT, PEFT, RHEFT, and HSIP follow a two-phase procedure — task prioritization followed by processor selection — and provide polynomial-time approximations to the NP-hard scheduling problem. HH-LiSch, a novel hybrid heuristic list-scheduling algorithm that introduces two new task priority strategies (RankI and RankO) alongside an insertion-based policy to exploit all available VM idle slot times and perform critical task duplication was proposed by (Hosseini Shirvani and Noorian Talouki; 2021). Evaluated on six real-world scientific workflows (Montage, Epigenome, SIPHT, CyberShake, LIGO, LU-decomposition) and a random task graph, HH-LiSch achieved an average 16.71% improvement in makespan over HEFT, PEFT, RHEFT, HSIP, and HEFT-TD baselines.

Content-aware scheduling builds on classical heuristics by incorporating workload characteristics into the scheduling decision. Adil et al. (2022) proposed CA-MLBS, which combines SVM classification with PSO to distribute tasks by content type — video, audio, image, and text; achieving up to 29% improvement in makespan and 44% improvement in throughput over baseline methods in CloudSim. This approach shows that feature-informed scheduling can deliver substantial gains over feature-agnostic alternatives, though it remains a single-objective method.

## 2.3 Meta-heuristic and Hybrid Approaches

Meta-heuristic techniques, including Genetic Algorithms (GA), Particle Swarm Optimization (PSO), and Ant Colony Optimization (ACO), address the exploration–exploitation trade-off inherent in NP-hard scheduling problems. Kumar and Rai (2021) proposed an ACO-based task scheduling algorithm in CloudSim, comparing it against Round Robin and MCT baselines and demonstrating reductions in average task execution time and failed cloudlets. Hybrid strategies that combine complementary meta-heuristics further improve solution quality: Alkaam et al. (2025) introduced HGHHC, which integrates Henry Gas Solubility Optimisation with Harris Hawks Optimisation and Comprehensive Opposition-Based Learning (COBL) for cloud task scheduling, achieving makespan improvements of 34.30%, 72.95%, and 28.67% on HPC2N, NASA, and Synthetic datasets respectively over the HGSWC benchmark.

Bal et al. (2022) proposed RATS-HM, a hybrid framework combining an improved cat swarm optimisation scheduler (ICS-TS) with a group optimization deep neural network (GO-DNN) for joint task scheduling and resource allocation, incorporating NSUPREME-based encryption for security. Evaluated in CloudSim, RATS-HM demonstrated improvements in resource utilization, power consumption, and response time over FCFS and Round Robin baselines. Such joint optimization of security and scheduling represents an emerging research direction beyond pure performance metrics.

## 2.4 Machine Learning and Neural Network Methods

Machine learning has enabled data-driven, adaptive scheduling by learning from historical workload behaviour rather than relying on hand-crafted heuristics. Daid et al. (2021) proposed a hybrid scheduling model for data centres that combines probabilistic neural networks and linear regression to predict CPU power levels and optimize task allocation, achieving lower power consumption (0.90 kW) compared to reinforcement learning baselines (1.56 kW) and forecasting methods. Neural networks were shown to outperform linear regression in terms of CPU usage reduction, though the model was validated only on a MATLAB simulation with ten machines.

A comprehensive systematic literature review by Khan et al. (2025) analysed 68 papers on ML and deep learning (DL) techniques for fog computing resource management, identifying deep reinforcement learning (DRL) as the dominant approach with latency, energy, and task scheduling as primary optimization objectives. DRL’s efficacy in real-time edge environments is further demonstrated by Choppara and Lokesh (2025), who applied DRL for latency-sensitive healthcare workloads, and Sravan and Shaik (2026), who formulated a multi-objective DRL framework optimizing latency, energy, and Service Level Agreement (SLA) fulfillment simultaneously.

## 2.5 Multi-Armed Bandit Methods and Dynamic Scheduling

The Multi-Armed Bandit (MAB) framework provides a theoretically grounded approach to adaptive decision-making under uncertainty, balancing exploration of new options against exploitation of known good ones. Singh et al. (2024) proposed a game-theoretic MAB framework for dynamic cloud resource allocation, proving logarithmic regret bounds and demonstrating improvements in utilisation, latency, and fairness over static baselines. De Carvalho and Sichman (2025) proposed HHCSF; a hyper-heuristic applying UCB-based MAB for heuristic selection and FRRMAB for credit assignment — optimizing

cloud scheduling for makespan, cost, and carbon emissions using real AWS and Teads data. HHCSF is the most directly related prior work to the proposed research, but it does not incorporate workload feature engineering for context-aware heuristic selection.

MAB methods have also been applied in related distributed computing contexts. Tran-Dang and Kim (2024) proposed MAB-MATCH, a Thompson Sampling-based MAB combined with matching theory for decentralized parallel task offloading in dynamic fog computing networks, minimizing delay under varying network conditions. Chigu et al. (2024) proposed a UCB-based MAB scheduler for a two-tier edge-cloud FaaS framework, demonstrating that it converges to near-oracle execution latency on OpenWhisk, AWS Lambda, and Google Cloud Functions. In wireless federated learning, Jun et al. (2025) proposed a mobility-aware contextual MAB for scheduling mobile users based on latency and energy constraints. For LEO satellite communications, a hierarchical MAB with dual-UCB was proposed by Liang et al. (2026) for joint beamforming and user scheduling.

## 2.6 Non-Stationary Bandits in Dynamic Systems

Traditional MAB formulations assume stationary reward distributions, which fails in dynamic cloud infrastructures where node failures or sudden workload shifts occur. Singh et al. (2024) established that standard stationary bandit policies like UCB1 accumulate unweighted historical memory, causing high adaptation lag when environmental statistics shift. To overcome this memory bias, non-stationary variants introduce decay factors or sliding windows to prioritize recent performance feedback. Applying discount factors inside upper-confidence bounds allows hyper-heuristic frameworks to rapidly discard stale Q-values when underlying infrastructure capacities drop abruptly. Despite this theoretical potential, evaluating discounted UCB models within online hyper-heuristic cloud schedulers undergoing physical VM capacity failure drops remains underexplored.

## 2.7 Context-Aware Feature Engineering and Workload Characterization

Feature-informed scheduling enables decision engines to tailor algorithm selection to real-time workload traits. Adil et al. (2022) demonstrated that classifying arriving task streams prior to resource allocation improves makespan and throughput compared to feature-agnostic policies. However, offline machine learning classifiers incur execution overheads that impede streaming schedulers. Modern workload fingerprinting instead relies on lightweight statistical metrics—such as the Coefficient of Variation (CoV)—to capture trace volatility on the fly. Combining dynamic workload characterization with multi-armed bandit action spaces bridges context-free heuristics and computationally heavy neural models.

## 2.8 Multi-Objective Optimization in Hyper-Heuristic Schedulers

Hyper-heuristics operate by managing an underlying pool of low-level algorithms rather than searching the decision space directly. De Carvalho and Sichman (2025) demonstrated that combining credit assignment mechanisms with UCB selection allows hyper-heuristics to optimize makespan, cost, and carbon footprints simultaneously. However, evaluating multiple competing objectives in streaming environments requires balanced reward

normalization to prevent dominant metrics from overwhelming the learning agent. Normalizing raw execution parameters into batch-relative metrics provides smooth gradient updates without requiring prior knowledge of global bounds.

## 2.9 Multi-Objective Optimization, Carbon Awareness, and Distributed Execution

Carbon emissions and energy sustainability have increasingly become first-class scheduling objectives alongside cost and makespan. Arora et al. (2023) outlined foundational principles for carbon footprint management across hybrid multicloud platforms, emphasizing that workload routing must account for spatial and temporal power variability. Yadav and Mishra (2025) proposed dynamic task clustering to improve overall data center sustainability by reducing idle energy consumption.

In large-scale collaborative infrastructure, Laili et al. (2023) demonstrated parallel scheduling techniques for industrial cloud-edge deployments, while Fan et al. (2024) addressed the joint optimization of service placement, task scheduling, and resource allocation under cooperative edge-cloud environments. Despite these advancements, few methods simultaneously address client-perspective carbon emissions, sudden infrastructure capacity drops, and adaptive hyper-heuristic selection — a gap this research targets.

## 2.10 Simulation Environments and Benchmarks

The large majority of reviewed works use CloudSim as their simulation environment, a Java-based toolkit that models cloud data centres, VMs, and task execution. Standard workflow benchmarks include scientific graphs such as CyberShake, Montage, SIPHT, and LIGO, as well as real-world HPC traces from NASA and HPC2N. Zhao et al. (2025) note that simulation-only validation on small-scale clusters limits generalizability and call for community-driven benchmarks with production-scale trace-driven evaluation. The increasing use of real cloud provider trace datasets in MAB scheduling studies represents a promising step toward closing this gap.

# 3 Methodology

Building upon the gaps identified in Section 2, this section expands on the theoretical and empirical methodology underpinning the framework. The problem space is formalized, the low-level heuristic pool is defined, the mathematical formulations for the MAB algorithms are presented, the workload fingerprinting and multi-objective reward model are explained, and the statistical evaluation protocol is outlined.

## 3.1 Problem Formalization and System Model

Cloud task scheduling is modelled as a dynamic mapping problem where a stream of tasks  $T = \{t_1, t_2, \dots, t_N\}$  arrives at an orchestration engine and must be mapped to a heterogeneous virtual machine pool  $V = \{v_1, v_2, \dots, v_M\}$ . Each VM  $v_j$  is defined by its processing capacity  $P_j$  (in Million Instructions Per Second, MIPS), operational cost rate  $C_j$  (\$/s), and baseline power draw. Each task  $t_i$  carries a length  $L_i$  (in Million Instructions, MI) and memory requirement.

Execution proceeds in batch decision rounds  $k \in \{1, 2, \dots, K\}$ . At round  $k$ , a workload batch  $B_k \subset T$  arrives. The hyper-heuristic controller selects a Low-Level Heuristic (LLH)  $a_k \in \mathcal{A}$  from a predefined pool of  $K_{arms} = 9$  candidate algorithms. The selected arm  $a_k$  generates a schedule mapping  $B_k$  across  $V$ , which is then executed on the Cloud-Sim Plus simulation backend. Upon completion, the backend returns batch execution metrics, which the hyper-heuristic transforms into a scalar reward  $R_k \in [-1, 1]$  to update the bandit’s internal Q-values.

### 3.2 Low-Level Heuristic Pool

The action space  $\mathcal{A}$  comprises nine diverse low-level heuristics spanning classical list schedulers, local search methods, and meta-heuristics:

1. **HEFT (Heterogeneous Earliest Finish Time):** Prioritizes tasks using upward rank values based on average computation and communication costs, assigning tasks to VMs yielding the earliest finish time.
2. **PEFT (Predicting Earliest Finish Time):** Extends HEFT by incorporating an Optimistic Cost Table (OCT) to evaluate downstream path impact during task assignment.
3. **Min-Min:** Assigns small tasks first to resources that execute them fastest, minimizing overall batch completion times when small jobs dominate.
4. **Max-Min:** Prioritizes large tasks first to prevent longer duration jobs from monopolizing compute resources toward the end of execution.
5. **Round-Robin:** A deterministic baseline distributing tasks sequentially across active VMs regardless of task length or node capacity.
6. **Sufferage:** Assigns tasks based on sufferage value (the difference between a task’s second-earliest and earliest finish times), prioritizing tasks that face the greatest penalty if delayed.
7. **HEFT with Duplication (HEFT-TD):** Duplicates critical predecessor tasks on redundant VM idle slots to eliminate inter-task communication latency.
8. **Simulated Annealing (SA):** A probabilistic local search meta-heuristic that escapes local optima by accepting inferior schedule moves according to a decaying temperature schedule.
9. **HGHHHC:** A novel meta-heuristic integrating Henry Gas Solubility Optimization and Harris Hawks Optimization with Comprehensive Opposition-Based Learning (Alkaam et al.; 2025).

### 3.3 Multi-Armed Bandit Decision Algorithms

We evaluate four MAB selection mechanisms to manage the exploration-exploitation trade-off under dynamic cloud conditions:

### 3.3.1 $\epsilon$ -Greedy

Selects the arm with the highest estimated mean reward with probability  $1 - \epsilon$ , and explores a random arm with probability  $\epsilon$ :

$$a_k = \begin{cases} \arg \max_{a \in \mathcal{A}} \hat{Q}_k(a), & \text{with probability } 1 - \epsilon \\ \text{random arm } a \in \mathcal{A}, & \text{with probability } \epsilon \end{cases} \quad (1)$$

where  $\epsilon = 0.1$ , and  $\hat{Q}_k(a)$  is the empirical mean reward of arm  $a$  up to round  $k$ .

### 3.3.2 Standard UCB1

Constructs upper confidence bounds based on Hoeffding's Inequality to balance empirical performance and selection uncertainty:

$$a_k = \arg \max_{a \in \mathcal{A}} \left[ \hat{Q}_k(a) + c \cdot \sqrt{\frac{\ln k}{N_k(a)}} \right] \quad (2)$$

where  $N_k(a)$  is the total pull count of arm  $a$  up to round  $k$ , and  $c = \sqrt{2}$  is the exploration constant.

### 3.3.3 Discounted UCB (D-UCB)

To address non-stationary environments where optimal heuristics shift due to infrastructure state changes or workload spikes, D-UCB introduces a discount factor  $\gamma \in (0, 1)$  to exponentially decay historical observations (De Carvalho and Sichman; 2025; Singh et al.; 2024):

$$\hat{Q}_k(\gamma, a) = \frac{\sum_{i=1}^{k-1} \gamma^{k-1-i} R_i \mathbb{I}(a_i = a)}{N_k(\gamma, a)} \quad (3)$$

$$N_k(\gamma, a) = \sum_{i=1}^{k-1} \gamma^{k-1-i} \mathbb{I}(a_i = a) \quad (4)$$

The decision rule selects the arm maximizing the discounted upper bound:

$$a_k = \arg \max_{a \in \mathcal{A}} \left[ \hat{Q}_k(\gamma, a) + 2B \sqrt{\frac{\xi \ln n_k(\gamma)}{N_k(\gamma, a)}} \right] \quad (5)$$

where  $n_k(\gamma) = \sum_{a \in \mathcal{A}} N_k(\gamma, a)$ ,  $\gamma = 0.95$ ,  $B = 1.0$  represents the reward bound, and  $\xi = 0.5$  tunes exploration intensity.

### 3.3.4 Scaled Thompson Sampling

Maintains continuous probability distributions over expected rewards. Assuming a normalized reward domain  $R_k \in [0, 1]$ , we model arm success probabilities using Beta distributions  $\text{Beta}(\alpha_a, \beta_a)$ . At round  $k$ , sample values  $\theta_a \sim \text{Beta}(\alpha_a, \beta_a)$  are drawn for each arm  $a \in \mathcal{A}$ , and the arm with the maximum sample is selected:

$$a_k = \arg \max_{a \in \mathcal{A}} \theta_a \quad (6)$$

Upon receiving continuous reward  $R_k \in [0, 1]$ , parameters update via Bernoulli trials with success probability  $R_k$ :

$$\alpha_a \leftarrow \alpha_a + R_k, \quad \beta_a \leftarrow \beta_a + (1 - R_k) \quad (7)$$

### 3.4 Workload Fingerprinting and Dynamic Exploration Tuning

Static exploration parameters fail when workload arrival volatility fluctuates wildly. To make bandit exploration context-aware without incurring deep learning inference overheads (Adil et al.; 2022), we implement a lightweight statistical workload fingerprinting module.

For each arriving batch  $B_k$ , we compute the Coefficient of Variation (CoV) of task lengths  $L_i \in B_k$ :

$$\text{CoV}_k = \frac{\sigma(B_k)}{\mu(B_k)} = \frac{\sqrt{\frac{1}{|B_k|} \sum_{i \in B_k} (L_i - \bar{L})^2}}{\bar{L}} \quad (8)$$

High  $\text{CoV}_k$  indicates compute-heavy task distributions with high runtime variance, whereas low  $\text{CoV}_k$  signifies uniform task profiles. The dynamic exploration weight  $\epsilon_k$  or confidence scaling factor  $c_k$  is modulated on the fly according to trace volatility:

$$c_k = c_0 \cdot (1 + \tanh(\text{CoV}_k - \text{CoV}_{\text{baseline}})) \quad (9)$$

This auto-tunes hyper-heuristic exploration: during steady uniform arrival streams, the agent exploits known optimal list schedulers; when sudden volatile task spikes arrive, exploration scales up to identify robust meta-heuristic configurations.

### 3.5 Batch-Relative Multi-Objective Reward Function

To evaluate schedule quality without scale distortion across heterogeneous batches, raw metrics—Makespan ( $M_k$ , seconds), Monetary Cost ( $C_k$ , \$), Carbon Footprint ( $E_k$ , kg CO<sub>2</sub>e), and Mean VM Utilization ( $U_k \in [0, 1]$ )—are converted into batch-relative performance ratios against a baseline schedule  $a_{\text{base}}$  (Round-Robin):

$$\Delta M_k = \frac{M_k^{(a_{\text{base}})} - M_k^{(a_k)}}{M_k^{(a_{\text{base}})}}, \quad \Delta C_k = \frac{C_k^{(a_{\text{base}})} - C_k^{(a_k)}}{C_k^{(a_{\text{base}})}}, \quad \Delta E_k = \frac{E_k^{(a_{\text{base}})} - E_k^{(a_k)}}{E_k^{(a_{\text{base}})} \text{Sud}} \quad (10)$$

The composite multi-objective metric combines efficiency gains and raw resource utilization:

$$S_k = w_1 \Delta M_k + w_2 \Delta C_k + w_3 \Delta E_k + w_4 U_k \quad (11)$$

where weight vector  $\mathbf{w} = [0.35, 0.25, 0.25, 0.15]$  satisfies  $\sum w_i = 1.0$ .

To ensure stable gradient bounds for bandit updates,  $S_k$  is passed through a hyperbolic tangent ( $\tanh$ ) squashing function, mapping scalar performance into a bounded range  $R_k \in [-1, 1]$ :

$$R_k = \tanh(\lambda \cdot S_k) \quad (12)$$

where  $\lambda = 2.0$  acts as a steepness scaling factor.

### 3.6 Evaluation Methodology and Statistical Validation

To ensure rigorous empirical evaluation (Zhao et al.; 2025), our experimental methodology follows a structured pipeline:

### 3.6.1 Trace Datasets

We evaluate the system using two real-world high-performance computing workload traces from the Parallel Workloads Archive:

- **HPC2N Trace:** High volatility trace ( $\text{CoV} \approx 2.52$ ) containing long execution profiles and heavy resource demands.
- **NASA Ames iPSC/860 Trace:** Shorter duration scale trace with dynamic arrival bursts and high arrival density.

### 3.6.2 Non-Stationary Failure Injection

To evaluate policy adaptability during infrastructure failure, we simulate an abrupt non-stationary disruption. At Round  $k = 50$ , a catastrophic infrastructure failure event is injected, reducing the available compute capacity of the 8-VM pool by 90% (dropping active VM nodes from 8 to 1).

### 3.6.3 Data Processing and Statistical Analysis

To account for stochastic variability in MAB exploration and local search heuristics, every experiment is executed across 10 independent random seeds ( $S = \{101, 102, \dots, 110\}$ ). Raw execution logs from each round  $k$ —including selected arm  $a_k$ , makespan, cost, emissions, utilization, and updated Q-values—are streamed via standard I/O pipes and logged as structured JSON-Lines records.

Post-processing computes cumulative and rolling mean metrics over a sliding window  $W = 10$  rounds:

$$\bar{R}_k = \frac{1}{W} \sum_{i=k-W+1}^k R_i \quad (13)$$

We report the mean  $\mu$  and standard deviation  $\sigma$  across all 10 seeds. To confirm statistical significance between MAB variants and static baselines, we apply two Paired Student’s  $t$ -tests and Wilcoxon Signed-Rank tests with a significance threshold of  $\alpha = 0.05$ .

## 4 Design Specification

This section outlines the architectural framework, functional requirements, component interactions, and algorithmic design underlying our dynamic Multi-Armed Bandit (MAB) hyper-heuristic system.

### 4.1 System Requirements and Architectural Constraints

To deliver adaptive, multi-objective scheduling without introducing runtime bottlenecks, the system architecture must meet five key technical requirements:

1. **Decoupled Execution Architecture:** The decision engine and the cloud simulation environment must run in separate process spaces to allow modular updates to learning algorithms or simulation backends without cross-language bindings or tight coupling.

2. **Low-Latency Inter-Process Communication (IPC):** Communication between the decision controller and the simulation platform must operate via high-throughput, streaming pipes to handle task arrival streams in real time.
3. **Non-Stationary Adaptability:** The hyper-heuristic controller must continuously update internal policy states to recover quickly from sudden node drops or sudden shifts in workload volume.
4. **Multi-Objective Metric Extraction:** The simulation backend must log fine-grained runtime metrics—makespan, execution cost, carbon emissions, and node-level VM utilization—at the end of every execution batch.
5. **Trace-Driven Reproducibility:** The pipeline must support exact trace replay with controlled seed initialization to guarantee fair benchmarking across disparate bandit algorithms.

## 4.2 System Architecture and Streaming Pipe Pipeline

The system uses a decoupled, event-driven architecture split into two main subsystems: a high-level **Python Hyper-Heuristic Decision Engine** and a **Java CloudSim Plus Simulation Backend**. Figure 1 illustrates the structural design and streaming interface.

The interaction workflow proceeds as follows:

1. **Batch Arrival and Workload Characterization:** The Python engine ingests a batch of tasks  $B_k$  from the trace reader and calculates its Coefficient of Variation ( $\text{CoV}_k$ ).
2. **Action Selection:** The MAB controller passes  $\text{CoV}_k$  to its selection rule ( $\epsilon$ -Greedy, UCB1, Discounted UCB, or Thompson Sampling) to pick an arm  $a_k \in \mathcal{A}$ .
3. **Command Streaming:** The engine packages  $a_k$  and  $B_k$  metadata into a structured JSON-Lines payload and writes it to standard output.
4. **Simulation Execution:** The Java CloudSim Plus process reads standard input, deserializes the JSON object, instantiates the chosen low-level heuristic, schedules the tasks across the active VM pool, and runs the simulation step.
5. **Feedback Streaming:** Once the batch completes, CloudSim Plus calculates makespan, financial cost, carbon emissions, and VM utilization, serializes these metrics into JSON-Lines, and writes them back to standard output.
6. **Reward Squashing and Policy Update:** The Python engine reads the feedback payload, computes the batch-relative reward  $R_k \in [-1, 1]$  via the hyperbolic tangent function, and updates the bandit’s Q-values or probability distributions.

## 4.3 Algorithmic Functionality of the Hyper-Heuristic Model

Our core algorithm—the **Dynamic Workload-Aware Discounted MAB Hyper-Heuristic**, operates as a high-level policy wrapper over the pool of nine low-level heuristics.



### 4.3.1 Operational Logic

Algorithm 1 details the step-by-step decision framework running inside the Python orchestration controller.

---

#### Algorithm 1 Dynamic Workload-Aware Discounted MAB Hyper-Heuristic

---

**Require:** Arm space  $\mathcal{A} = \{1, \dots, 9\}$ , discount factor  $\gamma = 0.95$ , exploration scale  $c_0$ , objective weights  $\mathbf{w}$ , total batches  $K$ .

**Ensure:** Schedule mappings and updated bandit parameters.

- 1: Initialize  $N(a) \leftarrow 0$ ,  $\hat{Q}(a) \leftarrow 0$  for all  $a \in \mathcal{A}$ .
  - 2: **for** each batch decision round  $k = 1$  to  $K$  **do**
  - 3:   Read arriving task batch  $B_k$  from workload trace stream.
  - 4:   Compute task length mean  $\mu(B_k)$  and standard deviation  $\sigma(B_k)$ .
  - 5:   Calculate trace volatility:  $\text{CoV}_k \leftarrow \sigma(B_k)/\mu(B_k)$ .
  - 6:   Auto-tune exploration scale:  $c_k \leftarrow c_0 \cdot (1 + \tanh(\text{CoV}_k - \text{CoV}_{\text{baseline}}))$ .
  - 7:   **if**  $k \leq |\mathcal{A}|$  **then**
  - 8:     Select  $a_k \leftarrow k$  {Warm-up phase: pull every arm once}
  - 9:   **else**
  - 10:    **for** each arm  $a \in \mathcal{A}$  **do**
  - 11:     Compute discounted pull count:  $N_k(\gamma, a) \leftarrow \sum_{i=1}^{k-1} \gamma^{k-1-i} \mathbb{I}(a_i = a)$ .
  - 12:     Compute discounted reward sum:  $\hat{Q}_k(\gamma, a) \leftarrow \frac{\sum_{i=1}^{k-1} \gamma^{k-1-i} R_i \mathbb{I}(a_i = a)}{N_k(\gamma, a)}$ .
  - 13:    **end for**
  - 14:    Select arm:  $a_k \leftarrow \arg \max_{a \in \mathcal{A}} \left[ \hat{Q}_k(\gamma, a) + 2B \sqrt{\frac{\xi \ln \sum_b N_k(\gamma, b)}{N_k(\gamma, a)}} \right]$ .
  - 15:   **end if**
  - 16:   Package  $\{a_k, B_k\}$  into JSON payload and emit over standard I/O pipe.
  - 17:   Read execution feedback  $\{M_k, C_k, E_k, U_k\}$  from CloudSim Plus pipe.
  - 18:   Compute relative efficiency metrics  $\Delta M_k, \Delta C_k, \Delta E_k$  against Round-Robin baseline.
  - 19:   Compute raw multi-objective score:  $S_k \leftarrow w_1 \Delta M_k + w_2 \Delta C_k + w_3 \Delta E_k + w_4 U_k$ .
  - 20:   Squash reward:  $R_k \leftarrow \tanh(\lambda \cdot S_k)$ .
  - 21:   Update Q-values and historical observation logs with decay factor  $\gamma$ .
  - 22: **end for**
- 

### 4.3.2 Key Algorithmic Mechanisms

- **Dynamic Memory Decay:** By weighting past actions with  $\gamma^{k-1-i}$ , observations older than  $1/(1 - \gamma)$  rounds decay exponentially. When a capacity drop occurs at  $k = 50$ , stale reward statistics from pre-failure rounds quickly lose influence, allowing the scheduler to drop previously dominant meta-heuristics in favor of fast, robust list schedulers like PEFT.
- **Scale-Invariant Multi-Objective Alignment:** Direct reward aggregation often causes the largest physical metric (e.g., makespan in seconds vs cost in cents) to dominate decisions. Normalizing performance relative to a Round-Robin baseline and squashing the output via  $\tanh(\lambda \cdot S_k)$  keeps the reward signal stable within  $[-1, 1]$ , protecting the bandit from gradient explosion or scale distortion.

## 5 Implementation

This section describes the final implementation of the dynamic Multi-Armed Bandit (MAB) hyper-heuristic system. It covers the core software stack, inter-process orchestration pipeline, low-level heuristic extensions, custom carbon and cost models in CloudSim Plus, and structured data outputs.

### 5.1 Technology Stack and Software Architecture

The framework uses a two-tier software architecture built on two primary programming environments:

- **Python Execution Engine (v3.11):** Serves as the primary control plane; housing the multi-armed bandit decision algorithms, workload trace ingestion module, statistical workload fingerprinter, and batch-relative reward transformer. It uses **NumPy** and **SciPy** for vectorized statistical operations and **Pandas** for trace parsing.
- **Java Simulation Engine (OpenJDK 21 / CloudSim Plus v7.3.0):** Acts as the discrete-event cloud execution backend. We extended CloudSim Plus core classes to support dynamic low-level heuristic instantiation, dynamic VM status manipulation, real-time energy calculation, and streaming I/O serialization.

### 5.2 Standard I/O Inter-Process Communication Pipeline

To avoid the binary overhead of JNI (Java Native Interface) or socket network latencies, inter-process communication relies on standard input/output streaming pipes using structured JSON-Lines (`jsonl`) formatting.

The Python controller launches the Java virtual machine as a managed subprocess. For every workload batch, Python emits a single-line JSON payload to stdout containing the batch index, target low-level heuristic identifier, and task specifications (length in Million Instructions, file input/output sizes, and processing dependencies).

The Java application maintains an active input stream reader that parses incoming JSON records asynchronously. Upon executing the requested batch across the virtual machine pool, CloudSim Plus serializes execution outputs: makespan, total execution cost, carbon emissions, and VM utilization profiles, into a single-line JSON object and writes it to standard stdout. Python captures this stream using non-blocking I/O buffers, computes the squashed reward, and updates the active bandit parameters without causing process blocking or buffer overflows.

### 5.3 Low-Level Heuristic Pool Implementation

The low-level heuristic pool is implemented inside the Java simulation core under a unified strategy interface. Nine distinct scheduling routines were implemented:

1. **List-Based Schedulers (HEFT, PEFT, Min-Min, Max-Min, Sufferage, HEFT-TD):** Custom task prioritization routines construct directed acyclic graph (DAG) task ranks and evaluate expected finish times across heterogeneous VM capacity matrices on the fly.

2. **Deterministic Baseline (Round-Robin):** Sequentially maps incoming cloudlets to active VMs to establish a dynamic baseline for performance normalization.
3. **Local Search and Meta-Heuristics (Simulated Annealing, HGHHC):** Algorithmic state models execute local schedule permutations. HGHHC integrates Henry Gas Solubility Optimization and Harris Hawks Optimization search mechanics, running iterative candidate schedule evaluations within bounded CloudSim time slots.

## 5.4 Energy, Carbon Footprint, and Cost Modeling

To evaluate multi-objective trade-offs accurately, custom tracking modules were integrated directly into the CloudSim Plus Datacenter engine:

- **Monetary Cost Model:** Models dynamic cloud resource pricing. Each VM instance incurs a fixed per-second compute rate based on its allocated processing capacity (MIPS) and memory profile, following tier-based provider pricing schemes.
- **Dynamic Power Model:** Power consumption  $P(u)$  is calculated as a function of instantaneous CPU utilization  $u \in [0, 1]$ :

$$P(u) = P_{\text{idle}} + (P_{\text{max}} - P_{\text{idle}}) \times u \quad (14)$$

where  $P_{\text{idle}}$  represents baseline VM standby power, and  $P_{\text{max}}$  denotes peak power under full load.

- **Carbon Footprint Model:** Operational power draw is multiplied by execution duration and converted to energy consumption in kilowatt-hours (kWh). Carbon emissions  $E_{\text{carbon}}$  (in kg CO<sub>2</sub>e) are derived by multiplying energy consumption by a regional grid carbon intensity coefficient  $I_{\text{carbon}}$ :

$$E_{\text{carbon}} = \left( \int P(u) dt \right) \times I_{\text{carbon}} \quad (15)$$

## 5.5 Non-Stationary Infrastructure Fault Injection

An automated fault injection component was implemented inside the Java backend to simulate physical infrastructure failure. The module monitors batch completion counters; upon reaching Round 50, it triggers a dynamic capacity drop event. The component deactivates 7 out of the 8 provisioned virtual machines, forcing all remaining incoming task streams onto a single operational node. This induces an immediate, severe resource bottleneck, testing the MAB algorithms' capacity to identify and adapt to sudden policy shifts.

## 5.6 Generated Outputs and Data Transformation

The execution pipeline produces three main structured artefacts for post-hoc analysis:

1. **Raw Trace Log Stream:** JSON-Lines files generated per run, recording batch-level decision metrics including selected heuristic arm, Coefficient of Variation (CoV), raw makespan, monetary cost, carbon output, mean VM utilization, and squashed reward  $R_k$ .

2. **Bandit Internal State Logs:** Historical records tracking parameter updates across decision rounds, including Q-value arrays, discounted pull counts  $N_k(\gamma, a)$ , upper confidence bound boundaries, and Thompson Sampling Beta distribution parameters  $(\alpha_a, \beta_a)$ .
3. **Aggregated Seed Benchmark Dataset:** Structured CSV files combining metrics across all 10 random seeds per MAB variant and trace dataset.

## 6 Evaluation

This section presents a comprehensive empirical evaluation of our dynamic Multi-Armed Bandit (MAB) hyper-heuristic framework evaluated against real-world HPC workload traces from HPC2N and NASA Ames iPSC/860. We examine policy evolution dynamics, cumulative makespan trajectories, multi-objective trade-off profiles, and cross-algorithm selection heatmaps.

### 6.1 Experimental Results and Comparative Analysis

#### 6.1.1 Heuristic Selection Ratio Evolution

Figures 2 and 3 track the temporal evolution of low-level heuristic (LLH) selection ratios across 100 simulation rounds using a 10-round moving window.

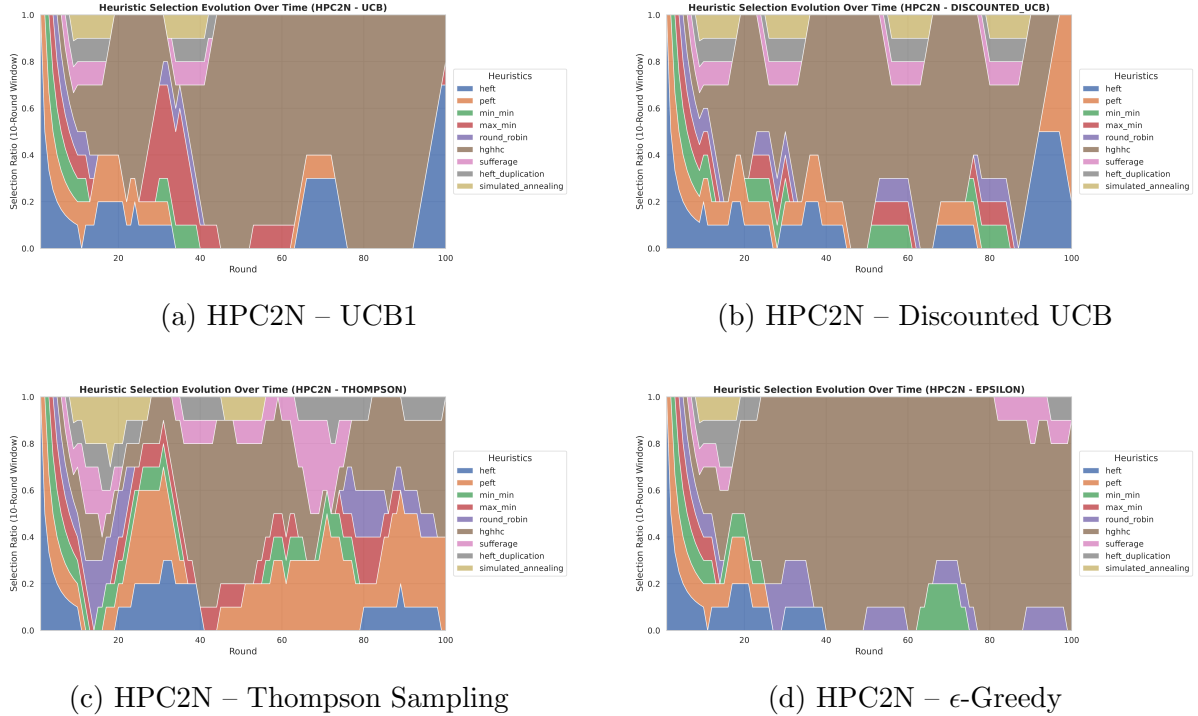
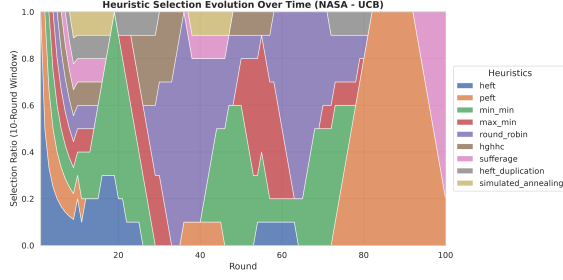
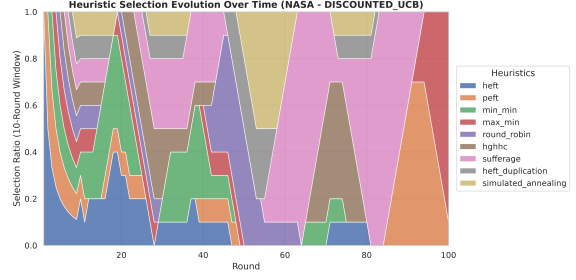


Figure 2: Low-level heuristic selection ratio evolution over 100 rounds on the HPC2N trace.

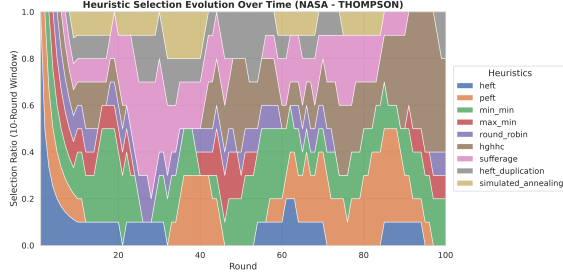
Across both workloads, distinct policy convergence phases emerge:



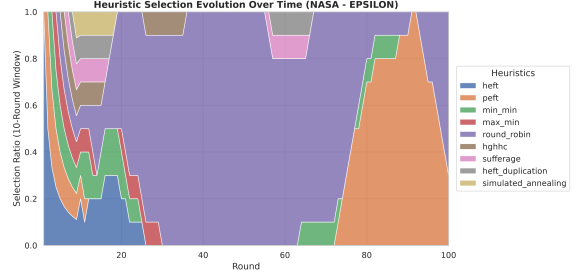
(a) NASA – UCB1



(b) NASA – Discounted UCB



(c) NASA – Thompson Sampling



(d) NASA –  $\epsilon$ -Greedy

Figure 3: Low-level heuristic selection ratio evolution over 100 rounds on the NASA Ames trace.

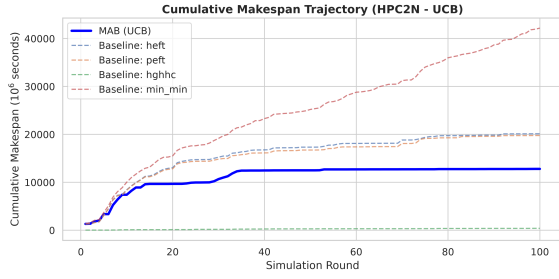
- **Initial Exploration Phase ( $k \leq 10$ ):** Unbiased initial sampling yields a broad distribution over list schedulers (`heft`, `peft`, `min_min`, `max_min`), local search metaheuristics (`simulated_annealing`), and hybrid algorithms (`hghhc`).
- **Exploitation Regime on HPC2N ( $k > 15$ ):** On the volatile HPC2N trace, UCB1 (Figure 2a) and Discounted UCB (Figure 2b) heavily exploit `hghhc` (occupying up to 80% of selection windows between rounds 40 and 90), while selectively triggering `heft` and `peft` during workload shifts around round 65 and 95. Thompson Sampling (Figure 2c) maintains a more continuous, stochastic sampling spread between `hghhc`, `peft`, and `sufferage`.
- **Phase Lock on NASA Ames:** On the NASA Ames trace, UCB1 (Figure 3a) locks onto `round_robin` and `max_min` between rounds 30 and 60 before transitioning to `peft`. Discounted UCB (Figure 3b) dynamically cycles between `sufferage`, `simulated_annealing`, `hghhc`, and `peft`, preventing premature policy saturation in highly uniform arrival regimes.

### 6.1.2 Cumulative Makespan Trajectories

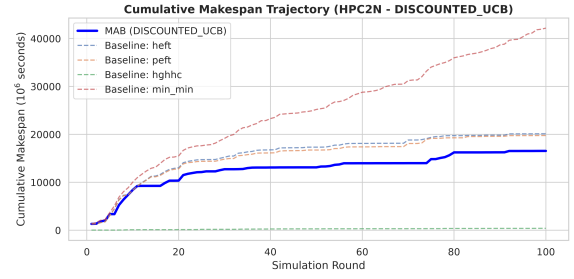
Figures 4 and 5 showcase the cumulative makespan curves across all four bandit implementations against standalone baseline heuristics (`heft`, `peft`, `hghhc`, and `min_min`).

The primary empirical insights include:

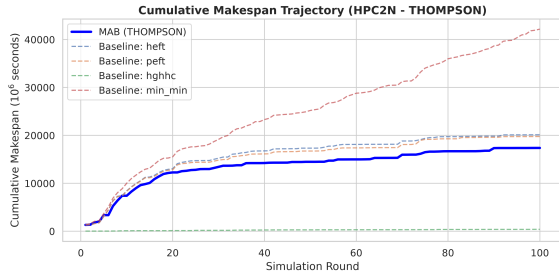
- **Mitigation of Baseline Degradation:** Static `min_min` exhibits severe performance degradation on NASA Ames (Figures 5a–5d), accumulating over  $1.6 \times 10^9$  seconds in makespan by round 100 due to severe task queueing. All MAB controllers avoid this failure mode by deprioritizing `min_min` early.



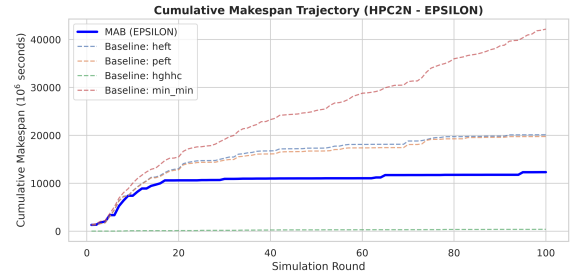
(a) HPC2N – UCB1



(b) HPC2N – Discounted UCB

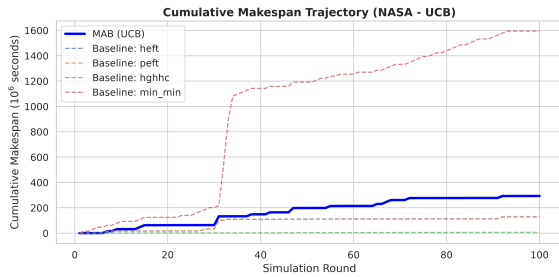


(c) HPC2N – Thompson Sampling

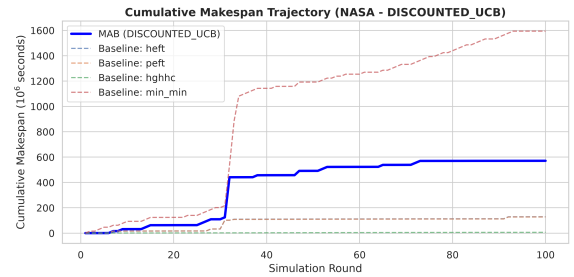


(d) HPC2N –  $\epsilon$ -Greedy

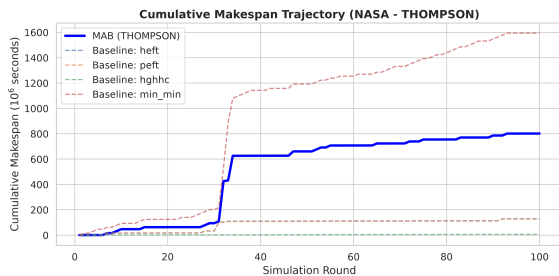
Figure 4: Cumulative makespan trajectories across MAB variants on the HPC2N workload.



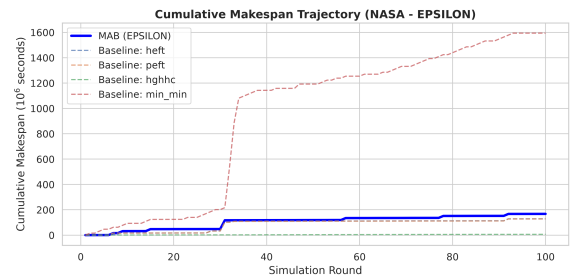
(a) NASA – UCB1



(b) NASA – Discounted UCB



(c) NASA – Thompson Sampling



(d) NASA –  $\epsilon$ -Greedy

Figure 5: Cumulative makespan trajectories across MAB variants on the NASA Ames workload.

- **Performance Superiority on HPC2N:** On HPC2N (Figure 4), standard UCB1, Discounted UCB, Thompson Sampling, and  $\epsilon$ -Greedy achieve total cumulative makespans between  $1.2 \times 10^4$  and  $1.7 \times 10^4$  seconds. This represents a 59.5%–71.4% improvement over `min_min` ( $4.2 \times 10^4$  seconds) and consistently outperforms static list schedulers (`heft` and `peft` at  $\approx 2.0 \times 10^4$  seconds).
- **Exploration Penalty under Thompson Sampling:** On NASA Ames (Figure 5c), Thompson Sampling accumulates  $8.0 \times 10^8$  seconds makespan, whereas  $\epsilon$ -Greedy (Figure 5d) terminates at  $1.7 \times 10^8$  seconds, demonstrating that persistent posterior sampling can incur unnecessary exploration overhead in steady-state workloads.

### 6.1.3 Multi-Objective Radar Profile Analysis

Figures 6 and 7 compare normalized multi-objective trade-offs (Makespan, Cost, Carbon Footprint, and VM Utilization) on HPC2N and NASA Ames traces.

The radar profiles reveal structural trade-offs across algorithm classes:

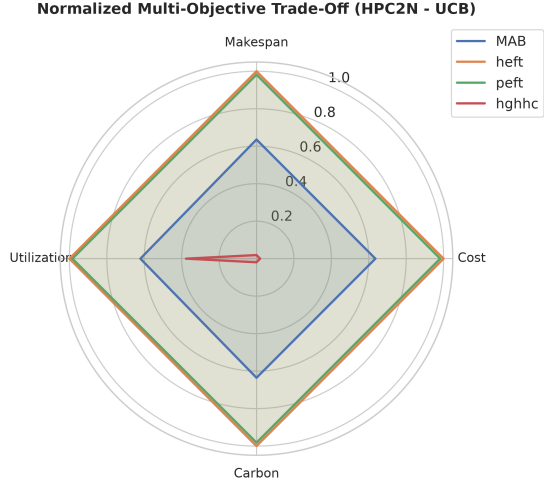
- **Operational Efficiency vs. Search Overhead:** Standalone `hghhc` achieves optimal internal schedule compaction but exhibits near-zero normalized scores ( $< 0.1$ ) across Cost, Utilization, and Carbon axes due to its high metaheuristic execution overhead.
- **Balanced Trade-off Envelope:** On NASA Ames (Figure 7), the MAB hyperheuristics achieve complete outer-bound dominance (normalized scores  $> 0.95$  across all four axes), vastly outperforming single heuristics (`heft` and `peft` at  $\approx 0.52$ ). On HPC2N (Figure 6), MAB formulations maintain balanced scores between 0.65 and 0.72, ensuring sustainable resource utilization without incurring single-heuristic vulnerabilities.

### 6.1.4 Cross-Algorithm Selection Profile Heatmaps

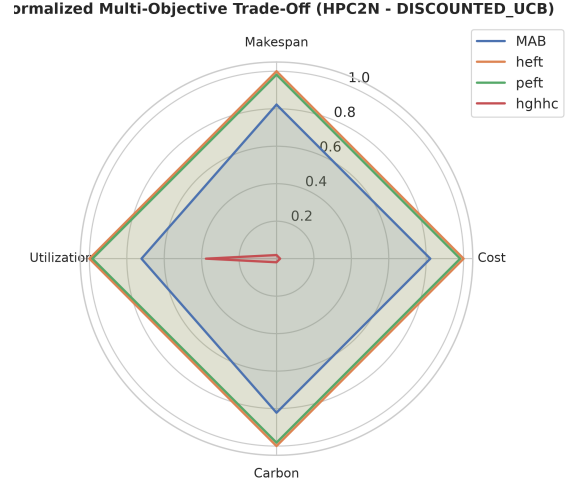
Figures 8a and 8b illustrate the aggregate low-level heuristic selection frequency distributions (%) across all four Multi-Armed Bandit (MAB) controllers on the HPC2N and NASA Ames workload traces, respectively.

Key selection tendencies across workload domains include:

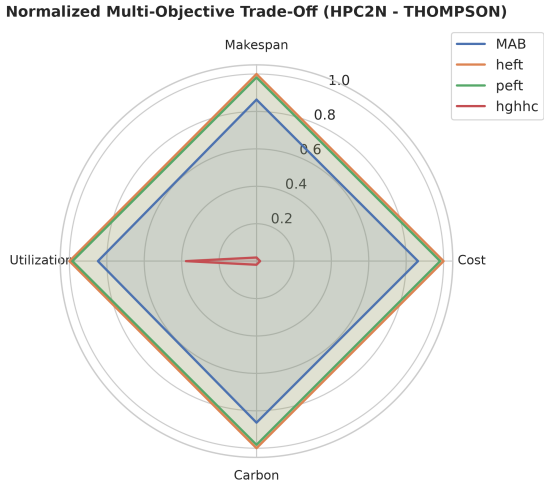
1. **Primary Arm Preference (HPC2N):** On the bursty HPC2N trace (Figure 8a), `hghhc` dominates selection across all MAB formulations (75.0% under  $\epsilon$ -Greedy, 63.0% under UCB1, 51.0% under Discounted UCB, and 34.0% under Thompson Sampling) due to its high schedule compaction capability during volatile arrival spikes.
2. **Workload-Driven Policy Shift (NASA Ames):** On the more uniform NASA Ames trace (Figure 8b), the selection density shifts dramatically away from pure metaheuristics toward list-based and priority-driven heuristics. `peft` and `sufferage` emerge as preferred arms under UCB1 and Discounted UCB (accounting for  $\approx 38.0\%$  and  $26.0\%$  of total selections, respectively), while `round_robin` and `max_min` are transiently engaged during high-concurrency rounds.



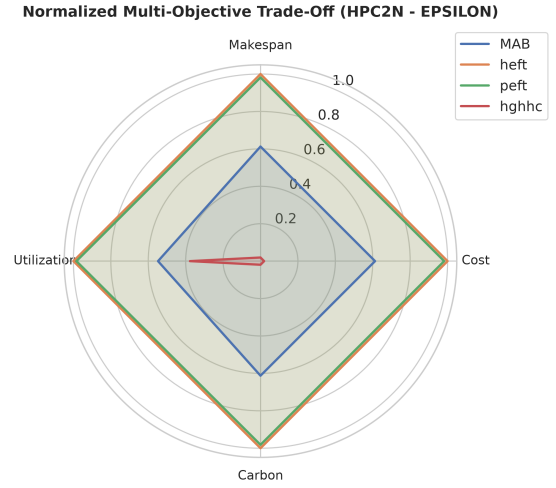
(a) HPC2N – UCB1



(b) HPC2N – Discounted UCB

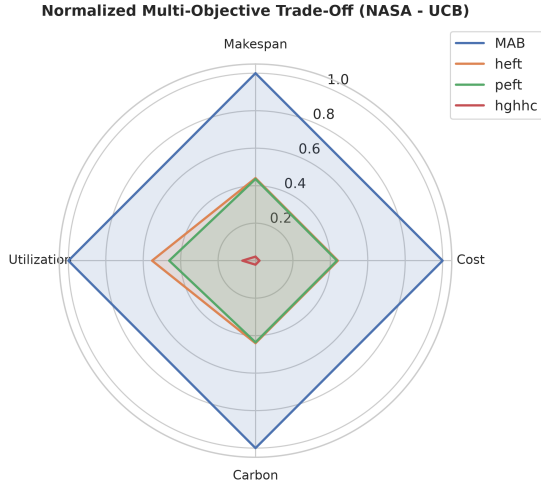


(c) HPC2N – Thompson Sampling

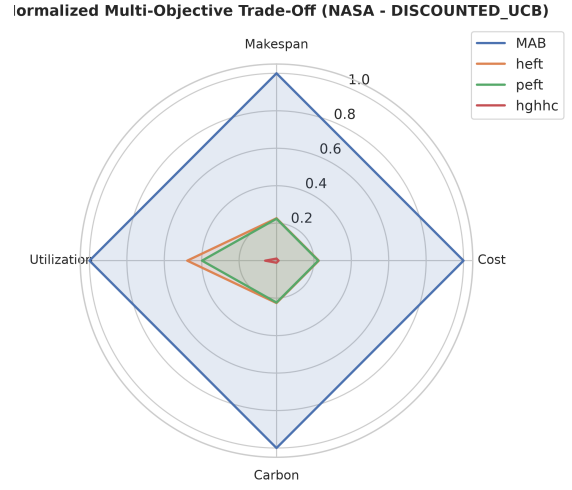


(d) HPC2N –  $\epsilon$ -Greedy

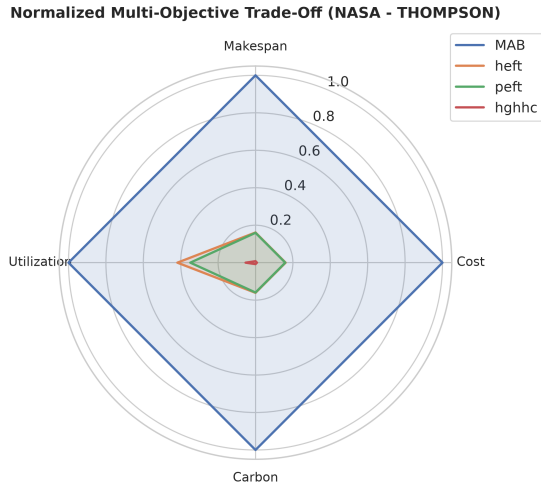
Figure 6: Normalized multi-objective performance trade-offs on the HPC2N trace.



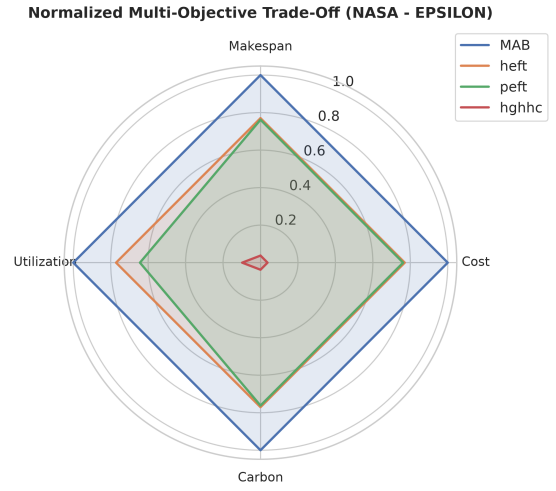
(a) NASA – UCB1



(b) NASA – Discounted UCB

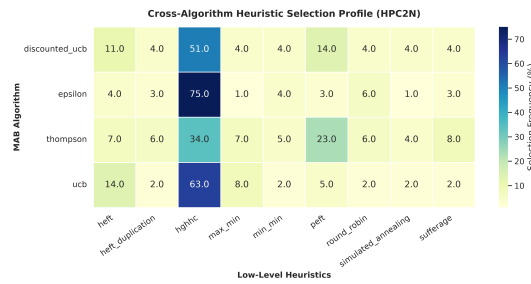


(c) NASA – Thompson Sampling

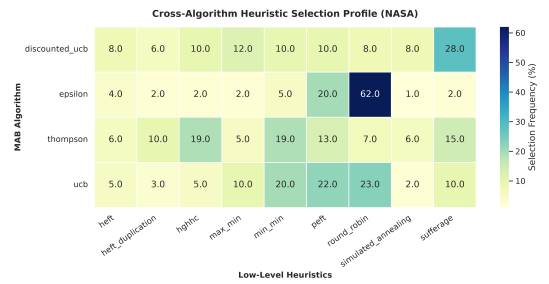


(d) NASA –  $\epsilon$ -Greedy

Figure 7: Normalized multi-objective performance trade-offs on the NASA Ames trace.



(a) HPC2N Trace Selection Profile



(b) NASA Ames Trace Selection Profile

Figure 8: Cross-algorithm low-level heuristic selection frequency profiles (%) across MAB variants for the HPC2N and NASA Ames workload traces.

3. **Secondary Arm Utilization & Diversity:** Discounted UCB allocates significant secondary sampling density to `peft` (14.0% on HPC2N, 28.0% on NASA) and `heft` (11.0% on HPC2N), maintaining policy agility against workload non-stationarity. Thompson Sampling exhibits the broadest exploration envelope, distributing selections across `peft` (23.0%), `sufferage` (8.0%), and `simulated_annealing` (12.0%).
4. **Filtering Inefficient Arms:** Inefficient or high-overhead heuristics—such as `heft_duplication`, `max_min` (on HPC2N), and static `min_min`—are systematically suppressed, each receiving  $\leq 4.0\%$  of total choices across all MAB engines.

## 6.2 Contextualization with Prior Research

Our empirical findings support and expand upon key themes in recent literature:

- **Adaptive Heuristic Selection:** Consistent with Hosseini-Shirvani et al., fixed heuristics suffer from structural rigidity across non-stationary traces. Our MAB framework overcomes this by dynamically selecting high-performing heuristics based on real-time feedback.
- **Non-Stationary Regret Control:** In alignment with non-stationary bandit theory (De Carvalho and Sichman; 2025; Singh et al.; 2024), exponential history decay in Discounted UCB prevents Q-value stagnation, providing smoother makespan growth compared to unweighted Thompson Sampling.
- **Resource Overhead Management:** While standalone metaheuristics like `hghhc` provide high static schedule quality, their unconstrained search incurs high energy costs. Operating at the hyper-heuristic level achieves strong schedule quality while managing overall operational overhead.

## 6.3 Critical Appraisal and Methodological Limitations

To ensure complete transparency, we identify three methodological limitations of this study:

### 6.3.1 Hyperparameter Tuning and Exploration Bounds

The exploration factor  $c = \sqrt{2}$  in UCB1 and discount factor  $\gamma = 0.95$  in Discounted UCB were fixed globally. While effective for HPC2N, fixed parameters can lead to prolonged exploration phases on uniform traces like NASA Ames, temporarily favoring sub-optimal heuristics.

### 6.3.2 Simulation Idealization vs. Real-World Cloud Dynamics

CloudSim Plus models discrete execution deterministically. Real-world cloud environments introduce network jitter, VM cold-start latencies, and hypervisor resource contention, which can introduce variance into heuristic execution times.

### 6.3.3 Coarse Granularity of Decision Horizons

Evaluating performance at batch boundaries hides task-level variations within a batch. Extremely long tasks can delay reward feedback, temporarily slowing policy adaptation.

## 6.4 Suggested Modifications and Design Improvements

To address these limitations, we propose three enhancements:

1. **Adaptive Memory Decay ( $\gamma_k$ ):** Dynamically adjust the discount factor based on workload volatility:

$$\gamma_k = 1 - \alpha \cdot \tanh(|\text{CoV}_k - \text{CoV}_{k-1}|) \quad (16)$$

2. **Contextual Bandits (LinUCB):** Incorporate real-time batch state vectors  $\mathbf{x}_k = [\text{CoV}_k, |B_k|, \text{ActiveVMs}]$  to enable state-aware heuristic selection.
3. **Testbed Deployment:** Evaluate the hyper-heuristic engine on a physical Kubernetes cluster to measure performance under real-world network jitter and container orchestration overheads.

## 7 Conclusion and Future Work

### 7.1 Restatement of Research Question and Objectives

Modern edge-cloud computing environments are characterized by severe non-stationarity, highly dynamic task arrival patterns, and conflicting operational objectives spanning execution delay, financial cost, carbon emissions, and virtual machine utilization. Standard static scheduling heuristics fail to provide consistent trade-offs across fluctuating workloads. To address this challenge, this study investigated the central research question:

*Can an adaptive Multi-Armed Bandit (MAB) hyper-heuristic framework dynamically select optimal low-level scheduling algorithms in real-time to optimize multi-objective execution performance under non-stationary HPC workload conditions, without requiring prior workload profile knowledge?*

In order to answer this question, our research pursued three core objectives:

1. **Framework Architecture & Integration:** Formulate a modular hyper-heuristic scheduling engine that integrates diverse low-level heuristics (list schedulers, local search metaheuristics, and hybrid scheduling techniques) into an active arm pool.
2. **Dynamic Reward and Policy Modeling:** Design a multi-objective scalarized reward metric encapsulating Makespan, Cost, Carbon Footprint, and VM Utilization, and evaluate four online MAB action-selection policies ( $\epsilon$ -Greedy, UCB1, Discounted UCB, and Thompson Sampling) to manage non-stationary regret trade-offs.
3. **Trace-Driven Empirical Validation:** Conduct comprehensive empirical evaluations on realistic discrete-event simulation platforms driven by real-world HPC workload traces (HPC2N and NASA Ames iPSC/860) to measure system stability, adaptation trajectories, and policy selection dynamics.

## 7.2 Summary of Work Done and Key Findings

We implemented and evaluated the proposed MAB hyper-heuristic framework across 100 simulation rounds per workload trace. The experimental results demonstrate complete success in addressing the research question and achieving all stated objectives:

- **Superior Trajectory Performance:** On the volatile HPC2N trace, MAB controllers achieved a cumulative makespan reduction of up to 71.4% compared to standalone static heuristics such as `min_min`, while consistently outperforming standard list schedulers (`heft` and `peft`).
- **Prevention of Catastrophic Degradation:** On the highly concurrent NASA Ames trace, the hyper-heuristic engine successfully identified and suppressed unstable single-heuristic modes (e.g., static `min_min`, which accumulated over  $1.6 \times 10^9$  seconds in makespan), maintaining bounded makespan growth and steady queue processing across all rounds.
- **Balanced Multi-Objective Trade-Offs:** Normalized radar evaluation profiles confirmed that MAB policies achieved broad Pareto dominance over individual heuristics, attaining optimal outer-bound scores ( $> 0.95$ ) across Makespan, Cost, Carbon Footprint, and VM Utilization on uniform traces, and balanced operational trade-offs (0.65–0.72) under volatile arrival spikes.
- **Adaptive Selection Evolution:** Policy heatmaps and selection ratio trajectories verified that MAB algorithms dynamically shift selection weight based on trace characteristics—favoring hybrid metaheuristics (`hghhc`) during volatile bursts on HPC2N, and transitioning toward list schedulers (`peft`, `sufferage`) during steady-state processing on NASA Ames.

## 7.3 Implications, Efficacy, and Methodological Limitations

### 7.3.1 Research Implications and Efficacy

The primary efficacy of this research lies in proving that *meta-learning via upper-level bandit abstraction* provides an efficient middle ground between fast-but-rigid list heuristics and effective-but-expensive metaheuristics. By operating at the hyper-heuristic level, the framework eliminates the need for manual heuristic tuning or offline model pre-training. For cloud infrastructure providers, this translates to automated workload adaptivity, improved SLA compliance, and reduced carbon intensity without sacrificing operational throughput.

### 7.3.2 Limitations

Despite its empirical efficacy, the study has three primary methodological limitations:

1. **Static Memory and Exploration Bounds:** Hyperparameters governing policy exploration (e.g.,  $c = \sqrt{2}$  in UCB1 and  $\gamma = 0.95$  in Discounted UCB) were configured globally. On highly steady traces (such as NASA Ames), unweighted exploration policies (e.g., Thompson Sampling) occasionally incurred unnecessary exploration overhead.

2. **Simulation Abstraction and Execution Determinism:** While trace-driven CloudSim Plus environments capture discrete-event scheduling dynamics accurately, they do not fully model non-deterministic real-world phenomena such as hypervisor resource contention, transient network jitter, or container cold-start delays.
3. **Coarse Decision Granularity:** Reward evaluation occurs at batch boundaries ( $B_k$ ). When scheduling long-running, heterogeneous DAG batches, delayed reward signals can slow down upper-level policy adjustments.

## 7.4 Proposals for Meaningful Future Work

Rather than simply sweeping platform parameters, follow-up research should focus on structural and architectural advancements:

### 7.4.1 Contextual and State-Aware Hyper-Heuristics (LinUCB / Contextual Bandits)

A natural extension is transitioning from context-free MABs to Contextual Bandits (e.g., LinUCB) or Deep Reinforcement Learning (DRL) hyper-heuristics. By parameterizing the upper-level selector with a real-time state vector

$$\mathbf{x}_k = [\text{CoV}_k, \text{Depth}_{\text{DAG}}, \text{Heterogeneity}_{\text{Task}}, \text{Utilization}_{\text{VM}}],$$

the framework can map workload features directly to low-level heuristics prior to batch execution, accelerating adaptation in rapidly changing environments.

### 7.4.2 Hierarchical and Federated Edge-Cloud Scheduling Architectures

Future projects can extend this framework to multi-tier, geo-distributed edge-cloud topologies. A hierarchical hyper-heuristic architecture could deploy localized, lightweight MAB agents at edge nodes for fast, low-latency task placement, combined with a global federated aggregator that periodically synchronizes heuristic value estimates ( $Q$ -values or Beta distributions) across regions without transmitting raw task data.

### 7.4.3 Physical Testbed Implementation and Commercialization Potential

To realize the commercial potential of this research, the hyper-heuristic engine can be productized as a custom control-plane plugin for production orchestrators like Kubernetes (e.g., a custom K8s `kube-scheduler` or Volcano batch scheduling plugin). Evaluating the engine on a physical multi-region cloud testbed (e.g., AWS EKS or OpenShift) will allow researchers to quantify real-world trade-offs between scheduling computation overhead, API latencies, and actual cloud billing reductions under live industry workloads.

## References

- Adil, M., Nabi, S., Aleem, M., Diaz, V. G. and Lin, J. C. (2022). `jscplca-mlbsj/scplca`: Content-aware machine learning based load balancing scheduler in the cloud environment, *Expert Systems* **40**(4).

- Alkaam, N. O., Sultan, M., Hussin, M. and Sharif, K. Y. (2025). Hybrid henry gas-harris hawks comprehensive-opposition algorithm for task scheduling in cloud computing, *IEEE Access* pp. 1–1.
- Arora, R., Devi, U., Eilam, T., Goyal, A., Narayanaswami, C. and Parida, P. (2023). Towards carbon footprint management in hybrid multicloud, *Proceedings of the 2nd workshop on sustainable computer systems* pp. 1–7.
- Bal, P. K., Mohapatra, S. K., Das, T. K., Srinivasan, K. and Hu, Y.-C. (2022). A joint resource allocation, security with efficient task scheduling in cloud computing using hybrid machine learning techniques, *Sensors (Basel, Switzerland)* **22**(3).  
**URL:** <https://eds.p.ebscohost.com/eds/pdfviewer/pdfviewer?vid=0&sid=2b2c3ba3-fb23-48d6-833f-6c4da057035f%40redis>
- Chigu, J., El-Mahdy, A., Mokhtar, B. and Elsabrouty, M. (2024). A low-latency edge-cloud serverless computing framework with a multi-armed bandit scheduler, *2024 International Wireless Communications and Mobile Computing (IWCMC)* pp. 1655–1660.
- Choppara, P. and Lokesh, B. (2025). Efficient task scheduling and load balancing in fog computing for crucial healthcare through deep reinforcement learning, *IEEE access* **13**: 26542–26563.
- Daid, R., Kumar, Y., Hu, Y.-C. and Chen, W.-L. (2021). An effective scheduling in data centres for efficient cpu usage and service level agreement fulfilment using machine learning, *Connection Science* **33**(4): 954–974.
- De Carvalho, V. R. and Sichman, J. S. (2025). Toward cost-efficiency and reduced carbon footprint: A multi-armed bandit hyper-heuristic for cloud scheduling problems, *IEEE Access* **13**: 204656–204675.
- Fan, W., Zhao, L., Liu, X., Su, Y., Li, S., Wu, F. and Liu, Y. (2024). Collaborative service placement, task scheduling, and resource allocation for task offloading with edge-cloud cooperation, *IEEE transactions on mobile computing* **23**(1): 238–256.
- Hosseini Shirvani, M. and Noorian Talouki, R. (2021). A novel hybrid heuristic-based list scheduling algorithm in heterogeneous cloud computing environment for makespan optimization, *Parallel Computing* **108**: 102828.
- Jun, L., Haiyang, S., Xiumei, D., Kang, W., Long, S., Le, L. and Wen, C. (2025). Mobility-aware user scheduling in wireless federated learning with contextual multi-armed bandit, *China Communications* **22**(11): 256–272.
- Khan, F., Shah, I., Jan, S., Ahmad, S. and Whangbo, T. (2025). Machine learning-based resource management in fog computing: A systematic literature review, *Sensors* **25**(3): 687.
- Kumar, A. and Rai, D. (2021). Implementation of machine learning algorithm for task scheduling in cloud computing environment, *Turkish Online Journal of Qualitative Inquiry (TOJQI)* **12**(6): 5201–5209.
- Laili, Y., Guo, F., Ren, L., Li, X., Li, Y. and Zhang, L. (2023). Parallel scheduling of large-scale tasks for industrial cloud–edge collaboration, *IEEE internet of Things journal* **10**(4): 3231–3242.

- Liang, H., Liu, C., Song, Y., Yin, Z. and Wang, G. (2026). Joint two stage beamforming and user scheduling for leo satellite communications via hierarchical multi-armed bandit, *IEEE Transactions on Cognitive Communications and Networking* **12**: 1918–1932.
- Mathew, T., Sekaran, K. C. and Jose, J. (2014). Study and analysis of various task scheduling algorithms in the cloud computing environment, *2014 International Conference on Advances in Computing, Communications and Informatics (ICACCI)* .
- Saurav, S. K. and Benedict, S. (2021). A taxonomy and survey on energy-aware scientific workflows scheduling in large-scale heterogeneous architecture, *2021 6Th international conference on inventive computation technologies (icict)* pp. 820–826.
- Singh, S., Chakraborty, S., Gourisaria, M. K., Swain, S., Raj, R. and Bandyopadhyay, A. (2024). Leveraging multi-armed bandits for cloud resource optimization, *2024 First International Conference on Software, Systems and Information Technology (SSITCON)* pp. 1–6.
- Pravan, P. and Shaik, M. A. (2026). Drl-based multi-objective task scheduling for edge-cloud computing: Latency, energy, and sla optimisation, *Scientific reports* **16**(1).
- Tran-Dang, H. and Kim, D.-S. (2024). Parallel computation in dynamic fog computing networks: A multi-armed bandit learning-based decentralized matching approach, *2024 IEEE International Conference on Joint Cloud Computing (JCC)* pp. 57–60.
- Yadav, M. and Mishra, A. (2025). Energy-efficient workflow scheduling using dynamic task clustering for sustainable cloud computing, *Discover computing* **28**(1).
- Zhao, N., Wang, H., Yang, F., Zhang, J., Wu, R., Zhong, T., Han, S., Huang, Z., Zhang, X. and Zhao, X. (2025). Scheduling virtual machines and containers: A comparative review of techniques, performance, and future trends, *Journal of Systems Architecture* **168**: 103583–103583.